

Figure 1: Active segments of memory

RAM.

- **Virtual address space** – The OS, with the help of sophisticated hardware, allows the running of virtual memory programs of almost unlimited size. This means even a very large file can be operated on by mapping it to the address space of the process without bothering the available RAM.
- **Memory protection** – A sophisticated memory management and hardware-assisted protection mechanism is supported under protected mode.
- **Multitasking** – Special instructions are available in protected mode that provide support to save the context of the current processor state and bring in the new task to the current context of the processor. The previous task can be brought back to the context and the execution starts from the point where it was left off in the past. All multi-tasking OSs exploit this feature based on their implementation of the policy of task context switching.

Protected mode: Segment translation

Segment translation is one of the key areas in *protected virtual address mode*. It is typically a mechanism of converting a *logical address* into a *linear address* by the *segmentation hardware* of the *MMU unit*. When a program gets loaded, the linker/loader loads the segmentation registers with the appropriate value. A protected mode segmentation register holds a 16-bit segment selector. And every segment selector has a linear *base address* associated with it, which is stored in the segment descriptor.

A 16-bit selector is used to store an index to access a segment descriptor in a descriptor table (descriptors were created by the linker/loader). Figure 2 shows the format of a selector.

As mentioned, the index field (bits [15:3]) has a value to access a descriptor in a descriptor table. This 13-bit index field is simply multiplied by the scale factor (i.e., the length

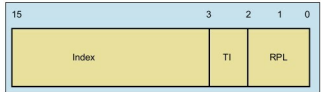


Figure 2: Format of a segment selector

of the descriptor, which is usually 8, as the descriptor requires 8-bytes in the table), and the result is then added to the base address of the descriptor table to get the desired segment.

1-bit Table Indicator or TI bit (refer to Figure 2) is an indication to determine which segment descriptor table has the segment. There are three segment descriptor tables that a processor can refer to:

- The Global Descriptor Table, or GDT,
- The Local Descriptor Table, or LDT, and
- The Interrupt Descriptor Table or IDT

If TI bit is set to 0, the segment reference is in GDT; otherwise it is in LDT. IDT is never referred by the segment selectors.

2-bit Requestor's Privilege Level or RPL bit (refer to Figure 2) is the indicator of the privilege level or ring. There are four privilege levels or rings – from 0 to 3. Each segment belongs to a ring with a designated privilege level. Operating system and kernel modules always run with privilege level 0, which has the highest priority. Applications that run in ring 3 have the least privileges.

To go over what we've covered, a selector is used to point a descriptor for the segment in a table of descriptors. The 32-bit base address (from the descriptor) is then added with the 32-bit offset to generate a 32-bit linear address.

If a paging unit is not active, the linear address generated by the segmentation hardware corresponds to the *physical address*, but if the paging unit is active, more needs to be done to generate the physical address; i.e., it requires a *paging translation*. Before looking at paging translation, let's briefly look at segment descriptor tables.

Segment descriptor tables

A logical address gets translated to a linear address with the use of the segmentation descriptor either from the *Global Descriptor Table (or GDT)*, or from the *Local Descriptor Table (or LDT)*.

Usually, only one GDT per system is permitted (global segments; in SMP, one GDT for every CPU in the system can be allowed); however, one LDT per process (or task) is defined, i.e., LDT segments are private to a specific program only. The *gdt* control register contains the address and the size of the GDT, whereas *ldt* is contained in the *ldtr* control register. The instructions *LGDT* (Load GDT) and *SGDT* (Store GDT) give access to GDT, while on the other hand, *LLDT* (Load LDT) and *SLDT* (Store LDT) give access to LDT in a system.

Any descriptor (GDT or LDT) would have the following main fields:

- **Base address** – Contains the linear (or virtual) address of the first byte of the segment.
- **Limit** – Contains the offset of the last memory cell in the